

A 3D rendering of a warehouse conveyor belt system. Several cardboard boxes of various sizes are positioned on the belt. Red laser lines form a grid pattern across the scene, suggesting a scanning or tracking system. The perspective is from a low angle, looking down the length of the conveyor.

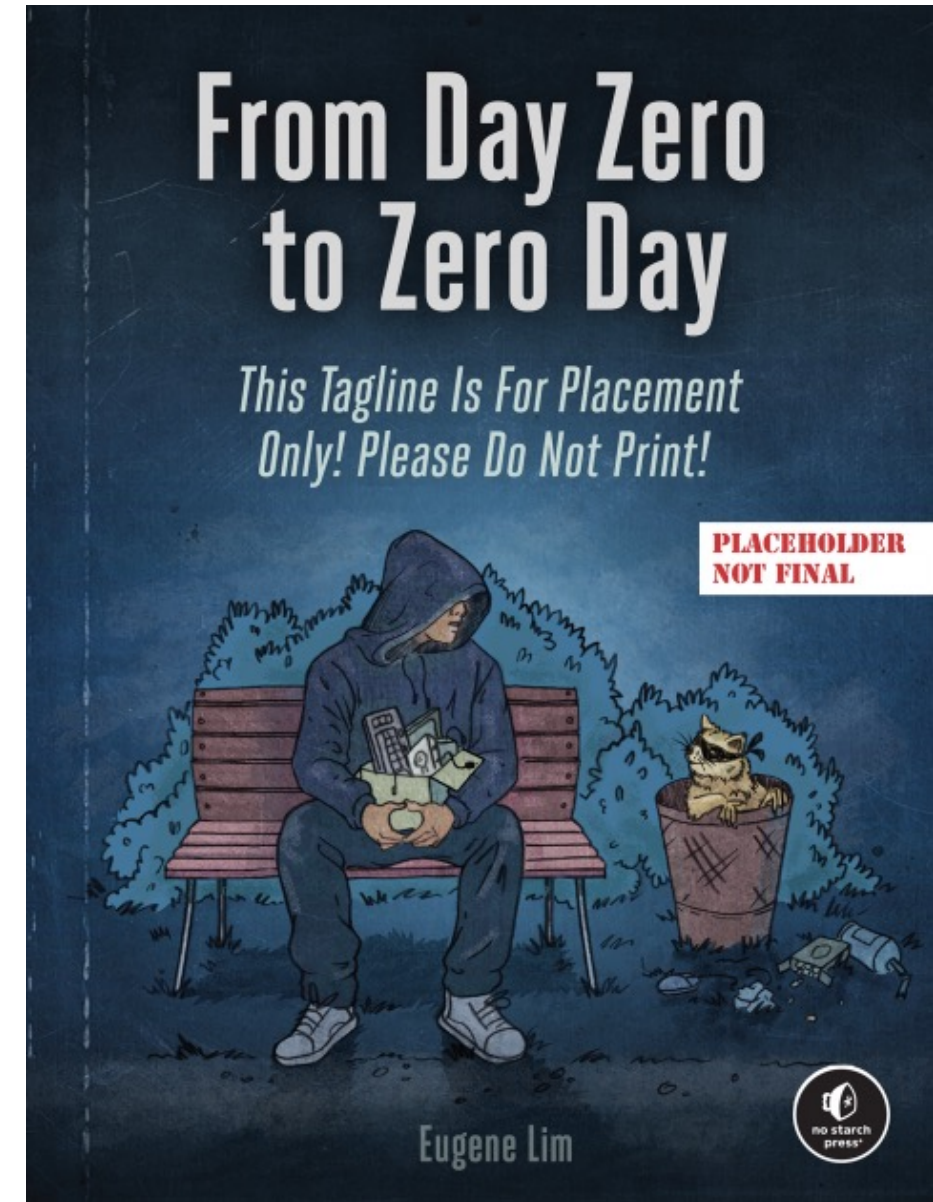
Pwning Millions of Smart Weighing Machines with API and Hardware Hacking

Eugene Lim

@spaceraccoon

hello world

- @spaceraccoon
- Lead Security Engineer, Open Government Products
- Hacking is my hobby
- “From Day Zero to Zero Day” published by No Starch Press, June 2025



What I'll be talking about today

My Goal: Introduce hardware hacking to software hackers!

A “how” and “why” process, not just a “what”.

- Distributed hardware authentication
- From machine authentication to user association
- Reverse-engineering mobile applications and firmware
- Exploiting association logic weakness

Overall Pick ⓘ



Etekcity Smart Scale for Body Weight Fat, FSA HSA Eligible, Digital Bathroom Weighing...

★★★★★ ∨ 208,063

6K+ bought in past month

\$79⁹⁹ (\$79.99/Count)

Delivery **Mon, Mar 17**

Ships to India

Best Seller



Etekcity Smart Scale for Body Weight FSA HSA Store Eligible, Bathroom Digital...

#1 Top Rated

★★★★★ ∨ 133,650

50K+ bought in past month

\$19⁹⁸ (\$19.98/Count)

Typical: \$22.99



Greater Goods Smart Weight Scale – Digital Bathroom Scale with BMI, Body Fat & Muscle...

★★★★★ ∨ 18,779

500+ bought in past month

Small Business ∨

See options

No featured offers available



Ultra, Smart Scale with 4.3" Color Display, FSA HSA Eligible, Digital Bathroom Scale for Body Weight BMI...

★★★★★ ∨ 24,682

2K+ bought in past month

See options

No featured offers available



Runstar Smart Scale for Body Weight and Fat Percentage, High Accuracy Digital...

★★★★★ ∨ 1,738

1K+ bought in past month

\$39⁹⁹ (\$39.99/Count)

List: \$54.99

Save \$5.00 with coupon

Let's look at the top-selling smart weighing machines today...

- WiFi or Bluetooth connectivity
- Configurable via mobile apps
- Kind of look the same...? (this will be important later)

How does hardware authentication and user association typically work?

Step One: Generate a unique ID/secret

 Static Identifier	 Keys	 Certs
• UUID • MAC Address • Serial Number	• Symmetric • Private-Public key pairs	• Certificate Signing Request • X.509 certificate

Concerns: Bruteforcing, Revocation, Identification

How does hardware authentication and user association typically work?

Step Two: Store it with the device



Packaging
(e.g. QR
code)



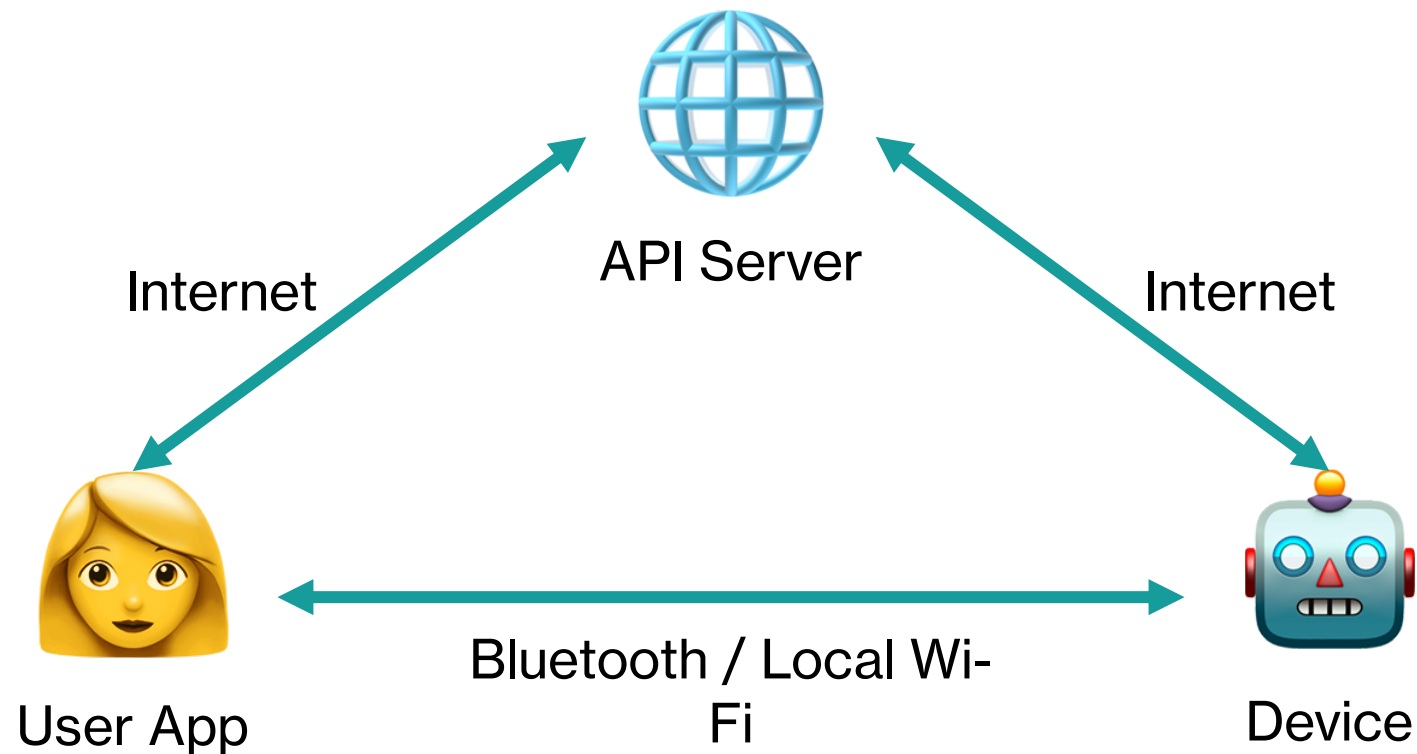
Hardware
storage



Secure
Element /
Trusted
Platform
Module

How does hardware authentication and user association typically work?

Step Three: Associate a user account with the hardware authentication



Why do all these mobile applications look similar?

Overall Pick 



Lepulse
Smart Weight Scale with BMI Body Fat Muscle Mass, Scale for Body Weight 4.68" Large Display with Trend, High...

★★★★★  72
300+ bought in past month

See options

No featured offers available
\$59.99 (1 new offer)



Scale for Body Weight and Fat, Lepulse Large Display Body Fat Scale, High Accura...

1 Count (Pack of 1)

★★★★★  4,337
1K+ bought in past month

See options

No featured offers available
\$49.99 (1 new offer)



BAGAIL Smart Scale for Body Weight, Digital Bathroom Scale for BMI Weighing Bod...

★★★★★  722
50+ bought in past month

See options

No featured offers available
\$20.99 (1 new offer)



RHINO
Smart Scale for Body Weight, High Precision, Bluetooth, Fitdays App, iOS and Android, Bathroom Wireless Machine...

★★★★★  2,495

See options

No featured offers available
\$19.99 (2 new offers)

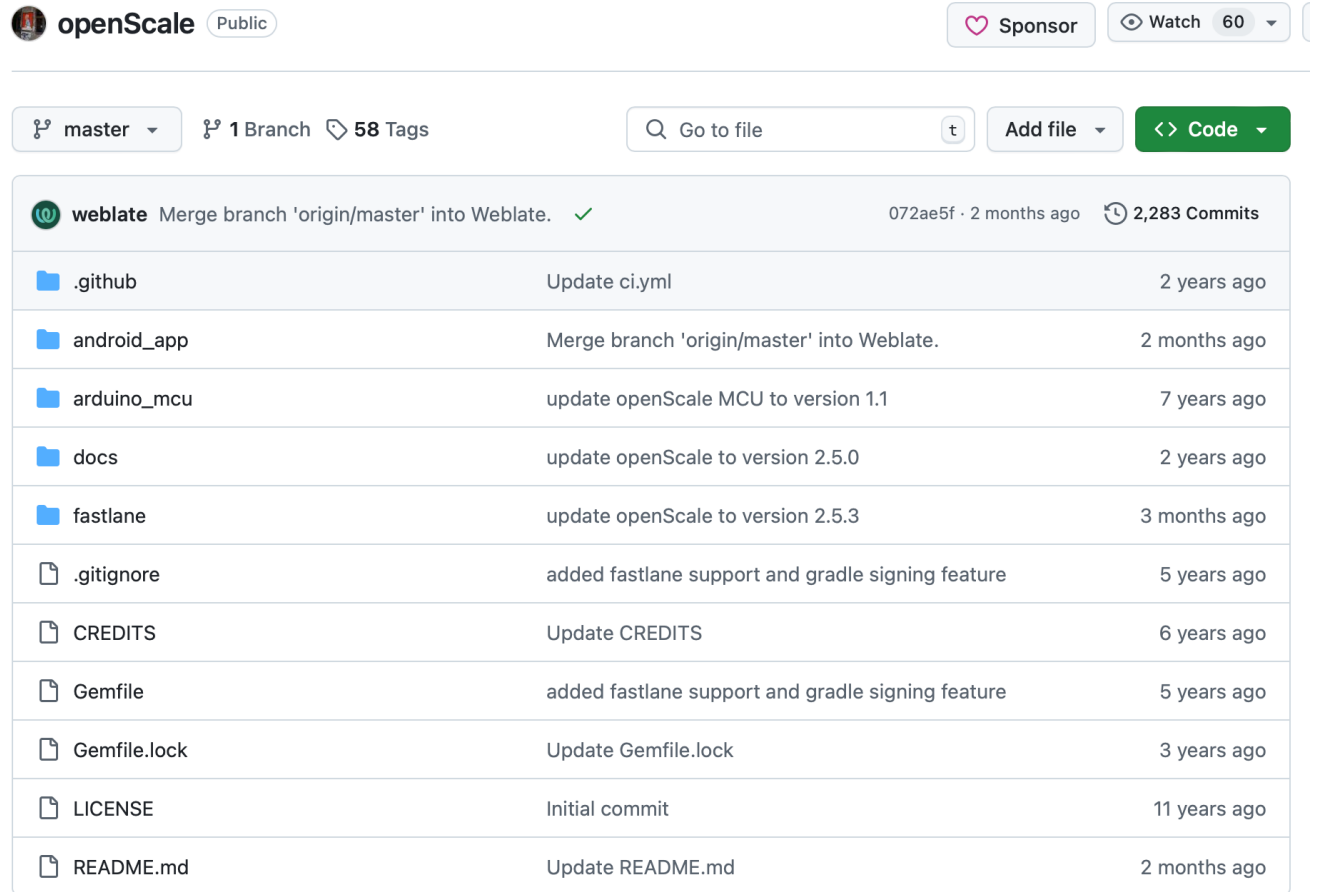
Different resellers use the same smart scale OEM's app, or even re-use code.

```
EXPLORER
sources > com > qingniu > heightscale > HeightMeasurePresenter.java
1 package com.qingniu.heightscale;
2
3 import android.content.Context;
4 import android.content.Intent;
5 import androidx.localbroadcastmanager.content.LocalBroadcastManager;
6 import com.qingniu.heightscale.constant.HeightScaleConstant;
7 import com.qingniu.heightscale.model.CompatHeightScaleConfig;
8 import com.qingniu.heightscale.model.HeightModeStatus;
9 import com.qingniu.qnble.blemanage.profile.BleProfileService;
10 import com.qingniu.scale.constant.DecoderConst;
11 import com.qingniu.scale.measure.MeasurePresenter;
12
13 /* loaded from: classes8.dex */
14 public class HeightMeasurePresenter extends MeasurePresenter {
15     public HeightMeasurePresenter(String str, Context context) {
16         super(str, context);
17     }
18
19     public void getRealTimeHeight(double d, HeightModeStatus heightModeStatus) {
20         Intent intent = new Intent(HeightScaleConstant.ACTION_REAL_TIME_HEIGHT);
21         intent.putExtra(BleProfileService.EXTRA_DEVICE_ADDRESS, d);
22         intent.putExtra(HeightScaleConstant.EXTRA_REAL_TIME_HEIGHT, heightModeStatus);
23         intent.putExtra(HeightScaleConstant.EXTRA_MODE_STATUS, heightModeStatus);
24         LocalBroadcastManager.getInstance(this.mContext).sendBroadcast(intent);
25     }
26
27     public void getRealTimeHeightFail(HeightModeStatus heightModeStatus, String message) {
28         Intent intent = new Intent(HeightScaleConstant.ACTION_REAL_TIME_HEIGHT_FAIL);
29         intent.putExtra(BleProfileService.EXTRA_DEVICE_ADDRESS, d);
30         intent.putExtra(HeightScaleConstant.EXTRA_REAL_TIME_HEIGHT, heightModeStatus);
31         intent.putExtra(HeightScaleConstant.EXTRA_MODE_STATUS, heightModeStatus);
32         intent.putExtra(message, message);
33         LocalBroadcastManager.getInstance(this.mContext).sendBroadcast(intent);
34     }
35 }
```

```
EXPLORER
sources > com > qingniu > heightscale > HeightMeasurePresenter.java
1 package com.qingniu.heightscale;
2
3 import android.content.Context;
4 import android.content.Intent;
5 import androidx.localbroadcastmanager.content.LocalBroadcastManager;
6 import com.qingniu.heightscale.constant.HeightScaleConstant;
7 import com.qingniu.heightscale.model.HeightModeStatus;
8 import com.qingniu.qnble.blemanage.profile.BleProfileService;
9 import com.qingniu.scale.constant.DecoderConst;
10 import com.qingniu.scale.measure.MeasurePresenter;
11
12 /* loaded from: classes2.dex */
13 public class HeightMeasurePresenter extends MeasurePresenter {
14     public HeightMeasurePresenter(String str, Context context) {
15         super(str, context);
16     }
17
18     public void getRealTimeHeight(double d2, HeightModeStatus heightModeStatus) {
19         Intent intent = new Intent(HeightScaleConstant.ACTION_REAL_TIME_HEIGHT);
20         intent.putExtra(BleProfileService.EXTRA_DEVICE_ADDRESS, d2);
21         intent.putExtra(HeightScaleConstant.EXTRA_REAL_TIME_HEIGHT, heightModeStatus);
22         intent.putExtra(HeightScaleConstant.EXTRA_MODE_STATUS, heightModeStatus);
23         LocalBroadcastManager.getInstance(this.f7864b).sendBroadcast(intent);
24     }
25
26     public void getRealTimeHeightFail(HeightModeStatus heightModeStatus, String message) {
27         Intent intent = new Intent(HeightScaleConstant.ACTION_REAL_TIME_HEIGHT_FAIL);
28         intent.putExtra(BleProfileService.EXTRA_DEVICE_ADDRESS, d2);
29         intent.putExtra(HeightScaleConstant.EXTRA_REAL_TIME_HEIGHT, heightModeStatus);
30         intent.putExtra(HeightScaleConstant.EXTRA_MODE_STATUS, heightModeStatus);
31         intent.putExtra(message, message);
32         LocalBroadcastManager.getInstance(this.f7864b).sendBroadcast(intent);
33     }
34 }
```

The openScale project focused on reverse-engineering the Bluetooth protocols...

- Use your own app to control multiple scales
- Home automation



The screenshot shows the GitHub repository page for 'openScale'. At the top, there's a header with the repository name 'openScale' (Public), a 'Sponsor' button, and a 'Watch' button with a count of 60. Below the header, there's a navigation bar with 'master' branch selected, '1 Branch', '58 Tags', a search bar 'Go to file', an 'Add file' button, and a 'Code' button. The main content area shows a commit history table for the 'weblate' user, listing recent merges and file updates.

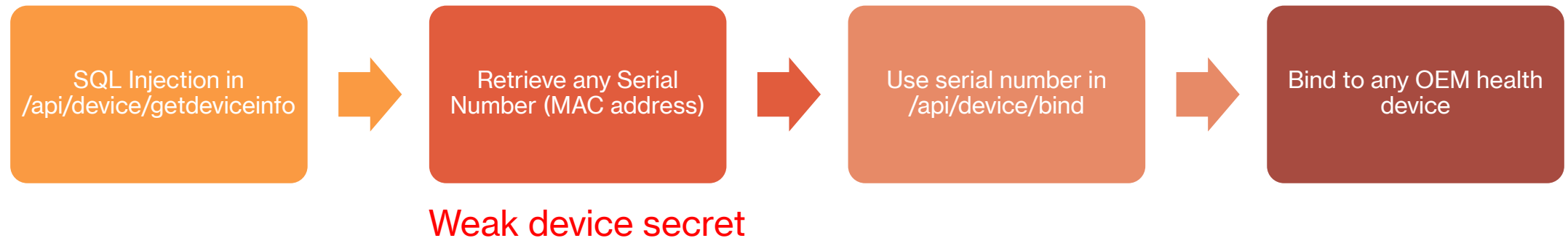
Commit Message	Commit Hash	Time Ago
Merge branch 'origin/master' into Weblate. ✓	072ae5f	2 months ago
Update ci.yml		2 years ago
Merge branch 'origin/master' into Weblate.		2 months ago
update openScale MCU to version 1.1		7 years ago
update openScale to version 2.5.0		2 years ago
update openScale to version 2.5.3		3 months ago
added fastlane support and gradle signing feature		5 years ago
Update CREDITS		6 years ago
added fastlane support and gradle signing feature		5 years ago
Update Gemfile.lock		3 years ago
Initial commit		11 years ago
Update README.md		2 months ago

...but for researchers, the server-side interactions are more interesting.

- Go beyond local proximity attacks to zero-interaction attacks
- Directly affects user-device associations at the API server

```
@Headers({"Content-type:application/json;charset=UTF-8"})
@POST("api/fitbit/bind")
Observable<BaseResponse<UserOperatingResponse>> bind(@Body RequestBody route);
@Headers({"Content-type:application/json;charset=UTF-8"})
@POST("api/device/bind")
Observable<BaseResponse<DeviceOperatingResponse>> bindDevice(@Body RequestBody
route);
@Headers({"Content-type:application/json;charset=UTF-8"})
@POST("api/device/bindv2")
Observable<BaseResponse<DeviceOperatingResponse>> bindv2(@Body RequestBody
route);
@Headers({"Content-type:application/json;charset=UTF-8"})
@POST("api/device/getDeviceInfo")
Observable<BaseResponse<DeviceOperatingResponse>> getDeviceInfo(@Body
RequestBody route);
@Headers({"Content-type:application/json;charset=UTF-8", "Domain-Name:
OTAUpdate"})
@POST("api/ota/update")
Observable<BaseResponse<OTAUpdateResponse>> getOtaUrl(@Body RequestBody route)
```

This netted my first vulnerability in an OEM application via a simple* SQL injection.



*bypass BT-WAF

Server-side APIs can also provide firmware files.

Sure beats trying to read it off
flash memory...

```
firmware_ver, String hardware_ver, final OnSuccessListener<OTAUpdateResponse>
listener) {
    if (SpHelper.getString("BaseUrl").contains("online-us")) {
        RetrofitUrlManager.getInstance().putDomain("OTAUpdate",
"https://<REDACTED>/");
    } else if (SpHelper.getString("BaseUrl").contains("online-eu")) {
        RetrofitUrlManager.getInstance().putDomain("OTAUpdate",
"https://<REDACTED>/");
    } else {
        RetrofitUrlManager.getInstance().putDomain("OTAUpdate",
"https://<REDACTED>/");
    }
    Gson gson = new Gson();
    HashMap hashMap = new HashMap(16);
    hashMap.put(AuthenticationTokenClaims.JSON_KEY_NAME,
org.apache.commons.lang3.StringUtils.defaultIfEmpty(name, "0"));
    hashMap.put("mac", org.apache.commons.lang3.StringUtils.defaultIfEmpty(mac,
"0"));
    hashMap.put("model",
org.apache.commons.lang3.StringUtils.defaultIfEmpty(model, "0"));
    hashMap.put("firmware_ver",
org.apache.commons.lang3.StringUtils.defaultIfEmpty(firmware_ver, "0"));
    hashMap.put("hardware_ver",
org.apache.commons.lang3.StringUtils.defaultIfEmpty(hardware_ver, "0"));
    hashMap.put(SocialConstants.PARAM_SOURCE, 0);
    BindInfo loadBindInfoByMac = GreenDaoManager.loadBindInfoByMac(mac);
    if (loadBindInfoByMac != null) {
        hashMap.put(AuthenticationTokenClaims.JSON_KEY_NAME,
loadBindInfoByMac.getName());
    }
    ...
}
```

Enter the Withings Body WBS06

- WiFi and Bluetooth Connectivity
- Custom mobile application
- Firmware pulled from server API



Withings
Smartphone
Tracker &

[Visit the Withings](#)

4.2 ★★★★★

₹5,999 (₹5,999)

Inclusive of all taxes
EMI starts at ₹2,999



Offers

Cashback

Upto ₹179.97
cashback as A
Pay Balance v

[1 offer >](#)

Reverse-engineering baremetal ARM firmware is hard. But some strings might help!

Connection Manager Shell Command

Usage:

```
wifi <wifi_sync_flags>
    Attempts a Wifi sync with the given flags.
    wifi_sync_flags is a combination of the
following flags:
    0x01 (allow update), 0x02 (store DbLib),
0x04 (send DbLib), 0x08 (send
    rawdata),
    0x10 (send wlog), 0x20 (send events), 0x40
(send extras)
    wifi_no_update <wifi_sync_flags>
    Attempts a Wifi sync, no update allowed (even if
set in flags).
    wifi_update <wifi_sync_flags>
    Attempts a Wifi sync, allows update if available
(even if not set in
    flags).
    bt    Attempts a Bluetooth sync
do    Attempts a Wifi/Cellular sync and fallback to
Bluetooth if it fails.
```



“Analyzing bare metal firmware binaries in Ghidra,” Attify Blog, Barun Kumar Basak

A screenshot of the Ghidra Memory Map window. The window title is 'Memory Map [CodeBrowser: Withings Body/wbs06_1181_XFUxfc.bin]'. It shows a table of memory blocks with columns for Name, Start, End, Length, Read (R), Write (W), Execute (X), Volatile, Overlay, Type, Initialized, Byte Source, Source, and Comment. The table lists three memory blocks: ram, sram, and peripheral.

Name	S...	End	Length	R	W	X	Volatile	Overlay	Type	Initialized	Byte Source	Source	Comment
ram	00000000	000db093	0xdb094	☒	☒	☒	☐	☐	Default	☒	File: wbs0...	Binary Lo...	
sram	1c000000	200fffff	0x4100000	☒	☒	☐	☐	☐	Default	☐			
peripheral	40000000	4007ffff	0x80000	☒	☒	☐	☐	☐	Default	☐			

But how do you go about accessing this shell?

A quick search pulls up this Reddit comment:



SureshotM6 · 1y ago

There are some people that have written tools to intercept the data being sent back to Withings in order to provide local logging. However, there's not a whole lot you can do with that.

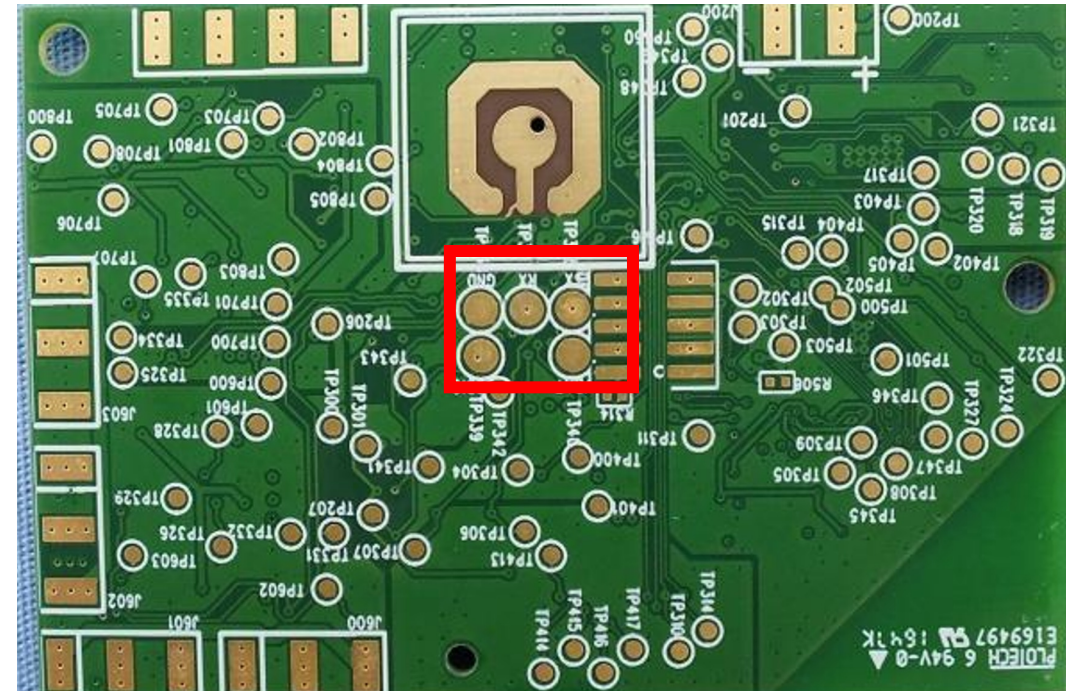
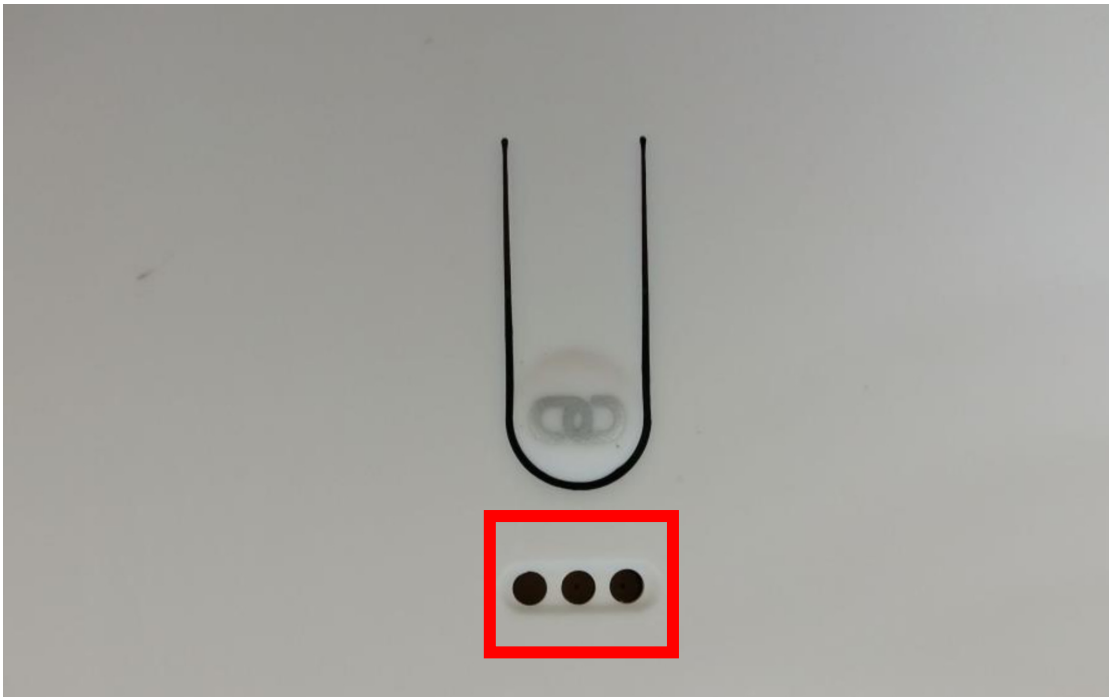
I have an older Body+ scale (WBS05). I took a look at it since I had already done some investigation into my Scanwatch a couple months ago. There is no firmware signing on the scale, just CRC32s. You could fairly easily modify the firmware to jump over (or NOP out) the draw commands for displaying the weight or for showing the Happy Birthday/New Year screens (which are easy to find). Then you'd need to update the CRCs in the firmware binary. There are 5 that would need updating on the Body+ that I see.

The harder part IMO would be getting the modified firmware back onto the device. If it supports firmware upgrade over Bluetooth like their watches, it shouldn't be too bad if you know their WPP protocol (it's fairly simplistic). Or if it always uses WiFi, you could intercept its traffic and provide your own binary. You may even be able to connect through the serial port on the bottom (those 3 holes are Tx, Rx, and GND) and load the firmware with some shell commands. Their devices provide a shell instead of normal operation when a connected UART is detected at power-up. It looks like it has 2 firmware partitions as well (similar to the Scanwatch). If its firmware selection is like the Scanwatch, you'd just need to make your version # higher.

Of course this whole thing comes with the risk of bricking your scale if you mess something up. At least there are redundant firmwares if it can detect an issue though... I'm happy enough with my scale that I'm not gonna modify it.

- Current target is WBS06
- Could it have the same serial port?
- Let's do a quick OSINT search before buying it...

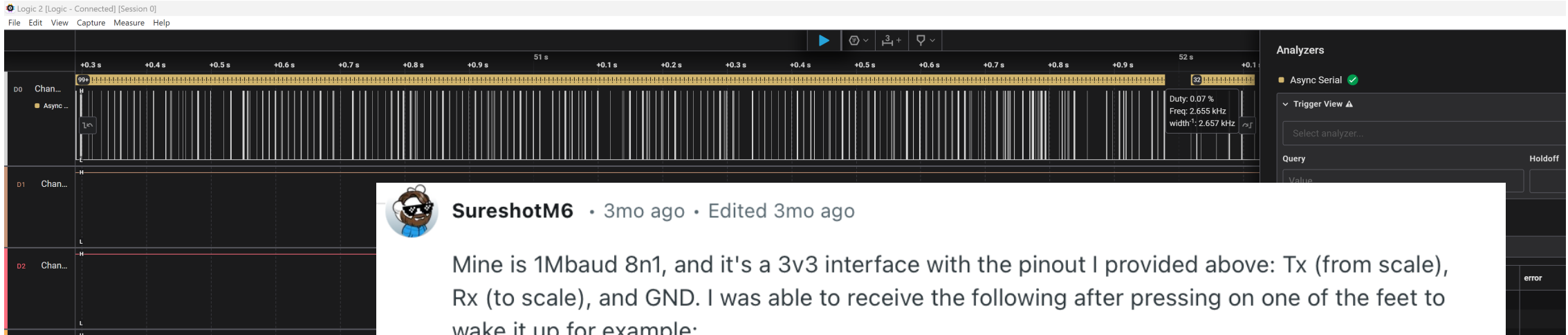
FCC certification documentation is a great way to get a peek into a device without opening it.



Getting the MCU also helps with reverse-engineering the firmware.



Trying to estimate the (non-standard) baud rate with poor equipment:



SureshotM6 · 3mo ago · Edited 3mo ago

Mine is 1Mbaud 8n1, and it's a 3v3 interface with the pinout I provided above: Tx (from scale), Rx (to scale), and GND. I was able to receive the following after pressing on one of the feet to wake it up for example:

Started with cheap CP2102,
ended up with FT232

```
-----> ENTERING MAINTASK <-----  
WDOG_RSTCNT=0  
[BAT] Use default cal  
[BAT] 6265 mv (100 %) (MIN 4450 MAX 6000) -> State <OK>  
[INITDBLIB] default bank valid  
[INITDBLIB] bank1 valid  
[INITDBLIB] bank2 valid  
[INITDBLIB] Bank courante: 1, address: 0x8000, size 0x8000  
[INITDBLIB] Bank autre: 2, address: 0x10000, size 0x8000
```

Demo



The WBS06 also had the same debugging shell!

- Multiple commands to read and write memory
- Read from DB, stored secrets
- Yay, mission accomplished...?

```
COM5 - PuTTY
CRC Ok
[CRC] FWBLK1 CRC OK
[CRC] FWBLK2 CRC OK
[CRC] INTERNAL FW CHECKING
[CRC] match fwblk 2: no
[CRC] match fwblk 1: yes
Reboot to NORMAL_BOOT
-----> ENTERING MAINTASK <-----
WDOG_RSTCNT=0
[BAT] Use default cal
[BAT] 6032 mv (100 %) (MIN 4450 MAX 6000) -> State <OK>
[INITDBLIB] default bank valid
[INITDBLIB] bank1 valid
[INITDBLIB] bank2 valid
[INITDBLIB] Bank courante: 1, address: 0x8000, size 0x8000
[INITDBLIB] Bank autre: 2, address: 0x10000, size 0x8000
[BAT] Use cal
[LOC] Stored locale: en_EN
RESET_REASON = 0x01
[info][DBLIB][SUBSADD] 119 0
[WFTL] Found 4 / 4 blocks for type 0
[WFTL] Found 11 / 11 blocks for type 2
[FLASH CACHE] Init ctx
[info]wlog_init
logs level display:2 store:2
EARLY_RESULT = First boot
*****UART IS CONNECTED*****

shell>
```

Let's recall two possible device-user association flows with certificates...

DEVICE

Present certificate to API server
corresponding to device ID

API server returns nonce

Sign nonce with local private key

Exchange signed nonce for
DEVICE session token

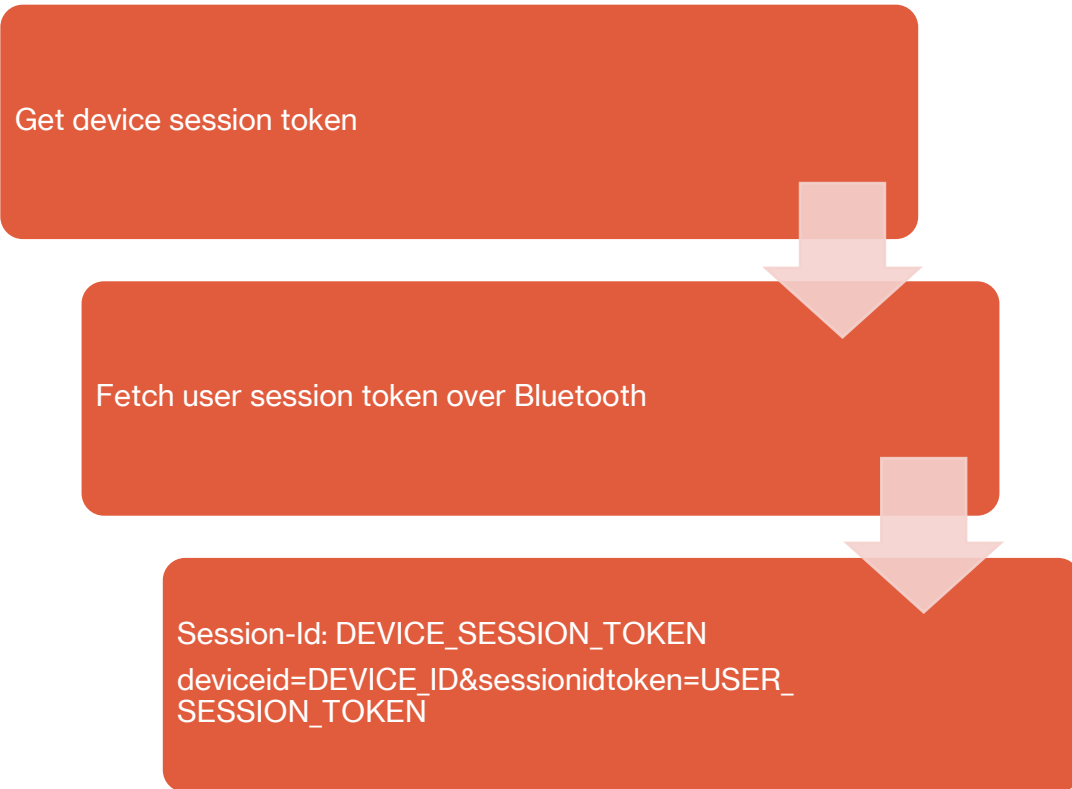
USER APP

Sign in with
username and
password

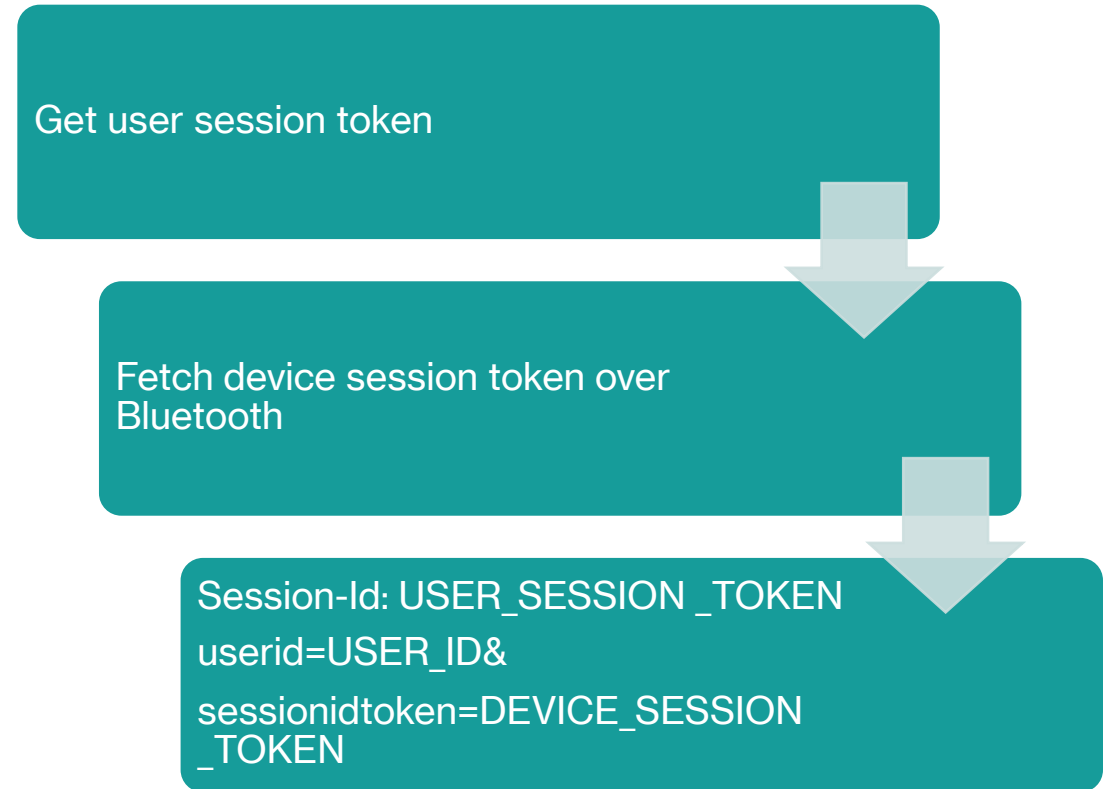
API server
returns USER
session token

For WBS06, device-user association could be done from the device or app.

DEVICE



USER APP



However, there was one fatal flaw in the business logic.

Session-Id: DEVICE_SESSION_TOKEN
deviceid=DEVICE_ID&
sessionidtoken=USER_SESSION_TOKEN

Both tokens are validated, DEVICE_ID and DEVICE_SESSION_TOKEN must correspond

Session-Id: USER_SESSION_TOKEN
userid=USER_ID&
sessionidtoken=DEVICE_SESSION_TOKEN

Both tokens are validated, USER_ID and USER_SESSION_TOKEN must correspond

Session-Id: USER_SESSION_TOKEN
deviceid=DEVICE_ID&sessionidtoken=
USER_SESSION_TOKEN

Both tokens are validated, USER_ID and USER_SESSION_TOKEN do not need to correspond

Associate with any device ID without a **DEVICE_SESSION_TOKEN!**

This time, the patch timeline was remarkably fast.



Takeaways

Device authentication has known best patterns...

...but user-device association can be tricky.

Hacking a single device might be fun, but what **about all of them?**

Methodology tips for a software-turned-hardware hacker



Understand your goal: Is it to get a shell? Test the authentication mechanism? Find injection vulns in the server-side API?



What will get you there? Map out the entire attack surface, from device to apps to the web, with reverse-engineering however you can.



Once you've found the sinks, work backwards to the source.

exit

- @spaceraccoon
- Lead Security Engineer, Open Government Products
- Hacking is my hobby
- H1-Elite Hall of Fam
- “From Day Zero to Zero Day” published by No Starch Press, June 2025

